Bölüm 8: Sınıflar ve OOP

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week4_Classes_OOP
7 Kasım 2026

İçindekiler

Bölüm 8 · Sınıflar ve Nesne Yönelimli Programlama 3

Bölümün amacı 3

Konu anlatımı 3

Prosedürel programlama (Slayt 1–3) 3

Nesne yönelimli programlama (Slayt 4–8) 3

Sınıf ve örnek (Slayt 9–11) 3

Sınıf tanımı, self ve __init__ (Slayt 12–15) 4

Gizli öznitelikler, parametreli metotlar ve __str__ (Slayt 16–18) 4

Örnek öznitelikleri (Slayt 19–21) 5

Accessor, mutator ve nesneyi argüman vermek (Slayt 22–23) 6

UML sınıf diyagramı (Slayt 24–26) 6

Problemdeki sınıfları bulmak (Slayt 27–32) 6

Sık yapılan hatalar 7

Kısa tekrar soruları 7

Cevaplar 8

Bölüm 8 · Sınıflar ve Nesne Yönelimli Programlama

Bölümün amacı

Bu bölüm veriyi ve o veri üzerinde çalışan kodu tek bir **nesnede** birleştirmeyi öğretir. Sonunda şunları yapabilmelisin:

- Prosedürel programlama ile nesne yönelimli programlamayı (OOP) karşılaştırmak; kapsülleme ve veri gizlemeyi açıklamak
- `class`, `__init__`, `self` ile sınıf tanımlamak ve örnek (instance) oluşturmak
- Private öznitelikler, accessor/mutator metotlar ve `__str__` yazmak
- UML sınıf diyagramını okumak; bir problem açıklamasından sınıfları ve sorumluluklarını çıkarmak

Konu anlatımı

Prosedürel programlama (Slayt 1–3)

Prosedürel programlama, belirli görevleri yapan fonksiyonlardan (prosedürlerden) oluşur. Veri, prosedürlerden ayrı durur ve prosedürden prosedüre aktarılır; odak, veri üzerinde çalışan prosedürlerdir. Veri yapısı değişince onu kullanan tüm fonksiyonların değişmesi gerekebilir.

Nesne yönelimli programlama (Slayt 4–8)

OOP nesneler oluşturmaya odaklanır. **Nesne**, hem veri hem prosedür içerir:

- Nesnenin verisine **veri öznitelikleri**, prosedürlerine **metotlar** denir.
- **Kapsülleme:** veriyi ve kodu tek bir nesnede birleştirmek.
- **Veri gizleme:** nesnenin verisine yalnızca kendi metotları erişir; veri kazara bozulmaz ve dışarıdaki kodun iç yapıyı bilmesi gerekmez.
- **Yeniden kullanım:** aynı nesne farklı programlarda kullanılabilir.

Bir saat nesnesinde saniye/dakika/saat **veri öznitelikleridir**; `set_time` gibi **public metotlar** dışarıya açıktır, iç işleyiş için **private metotlar** kullanılır.

Sınıf ve örnek (Slayt 9–11)

Sınıf, bir nesne tipinin özniteliklerini ve metotlarını tanımlayan koddur (ev planı, kurabiye kalıbı). **Örnek** (instance) o sınıftan oluşturulmuş belirli bir nesnedir. Bir sınıfın birçok örneği olabilir.

Sınıf tanımı, self ve init (Slayt 12–15)

- `class SinifAdi:` ile başlar; ad genellikle büyük harfle yazılır.
- Her metodun ilk parametresi `self`'tir ve metodun üzerinde çalıştığı nesneyi gösterir.
- `__init__` başlatıcıdır: örnek oluşturulurken otomatik çalışır ve öznitelikleri başlatır.
- Örnek: `nesne = SinifAdi()` · metot çağrısı: `nesne.metot()`; `self` otomatik aktarılır.

PYTHON

```python
import random
random.seed(3)

class Coin:
    def __init__(self):
        self.sideup = 'Heads'

    def toss(self):
        if random.randint(0, 1) == 0:
            self.sideup = 'Heads'
        else:
            self.sideup = 'Tails'

    def get_sideup(self):
        return self.sideup

my_coin = Coin()
print('Başta:', my_coin.get_sideup())
for _ in range(4):
    my_coin.toss()
    print(my_coin.get_sideup(), end=' ')
```

ÇIKTI

```
Başta: Heads
Heads Heads Tails Tails
```

Gizli öznitelikler, parametreli metotlar ve str (Slayt 16–18)

Adı **iki alt çizgiyle** başlayan öznitelik (`self.__balance`) private olur ve dışarıdan doğrudan erişilemez. `__init__` ve diğer metotlar `self`'e ek parametreler alabilir. Bir nesnenin **durumu**, özniteliklerinin o anki değerleridir; `__str__` durumu anlatan bir string döndürür ve nesne `print` ya da `str` ile kullanılınca otomatik çağrılır. Sınıflar modüllerde (`.py`) saklanıp import edilebilir.

PYTHON

```python
class BankAccount:
    def __init__(self, bal):
        self.__balance = bal

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if self.__balance >= amount:
            self.__balance -= amount
        else:
            print('Error: Insufficient funds')

    def get_balance(self):
        return self.__balance

    def __str__(self):
        return f'The balance is ${self.__balance:,.2f}'

hesap = BankAccount(1000.0)
hesap.deposit(500)
hesap.withdraw(2000)
print(hesap.get_balance())
print(hesap)
```

ÇIKTI

```
Error: Insufficient funds
1500.0
The balance is $1,500.00
```

Örnek öznitelikleri (Slayt 19–21)

Bir metot `self.ad = ...` ile öznitelik oluşturduğunda bu **örnek özniteliğidir**: her örneğin **kendi** kopyası vardır. Bir nesneyi değiştirmek diğerlerini etkilemez.

PYTHON

```python
class Coin:
    def __init__(self):
        self.sideup = 'Heads'

coin1 = Coin()
coin2 = Coin()
coin1.sideup = 'Tails'
print(coin1.sideup, coin2.sideup)
```

ÇIKTI

```
Tails Heads
```

Accessor, mutator ve nesneyi argüman vermek (Slayt 22–23)

Öznitelikler genellikle private tutulur ve metotlarla erişilir:

- **Accessor** (erişici): değeri değiştirmeden döndürür (`get_balance`).
- **Mutator** (değiştirici): değeri saklar veya değiştirir (`set_balance`, `deposit`).

Bir nesneyi fonksiyona vermek ona bir **referans** vermektir: fonksiyon asıl nesnenin metotlarını çağırıp durumunu değiştirebilir.

PYTHON

```
class Coin:
    def __init__(self):
        self.__sideup = 'Heads'
    def set_tails(self):
        self.__sideup = 'Tails'
    def get_sideup(self):
        return self.__sideup

def cevir(coin_obj):
    coin_obj.set_tails()

c = Coin()
cevir(c)
print(c.get_sideup())
```

ÇIKTI

```
Tails
```

UML sınıf diyagramı (Slayt 24–26)

UML'de bir sınıf üç bölmeli bir kutudur: üstte **sınıf adı**, ortada **veri öznitelikleri**, altta **metotlar**.

Coin
`__sideup`
`__init__()`, `toss()`, `get_sideup()`

Problemdeki sınıfları bulmak (Slayt 27–32)

1. Problem alanının yazılı açıklamasını al.
2. Açıklamadaki tüm **isimleri** (isim tamlamaları, zamirler) belirle; her biri potansiyel sınıftır.
3. Listeyi **daralt**: aynı anlama gelenleri, programın ilgilenmediği öğeleri, sınıf değil nesne olanları ve basit bir değişkene sığan değerleri çıkar.

Sınıfın **sorumlulukları**: bilmesi gerekenler → veri öznitelikleri; yapması gerekenler → metotlar.

Sık yapılan hatalar

- **Metotta `self` parametresini unutmak:** çağrıda Python nesneyi otomatik gönderdiği için `TypeError` olur.

PYTHON
```
class Sayac:
    def __init__(self):
        self.n = 0
    def artir():
        self.n += 1

Sayac().artir()
```

ÇIKTI
```
TypeError: artir() takes 0 positional arguments but 1 was given
```

- **Private özniteliğe dışarıdan erişmeye çalışmak:** `hesap.__balance` → `AttributeError`; accessor kullan.

PYTHON
```
class Hesap:
    def __init__(self, b):
        self.__bakiye = b
print(Hesap(100).__bakiye)
```

ÇIKTI
```
AttributeError: 'Hesap' object has no attribute '__bakiye'
```

- **Özniteliği `self.` olmadan atamak:** `__init__` içinde `n = 0` yerel değişken olur, öznitelik oluşmaz.
- **Sınıfla örneği karıştırmak:** `Coin.toss()` değil, `my_coin = Coin()` sonra `my_coin.toss()`.

Kısa tekrar soruları

1 Kapsülleme ve veri gizleme ne demektir?

2 `__init__` ne zaman çalışır?

3 `self` parametresi neyi gösterir?

4 Accessor ve mutator metot örneği ver.

5 `print(nesne)` yazınca hangi metot otomatik çağrılır?

6 Bir problem açıklamasından sınıf bulurken hangi isimler listeden çıkarılır?

Cevaplar

1 Kapsülleme: veri ve kodu tek nesnede birleştirmek. Veri gizleme: verinin dışarıdan gizlenip yalnızca nesnenin metotlarıyla erişilebilmesi.

2 Sınıftan bir örnek oluşturulduğu anda otomatik olarak.

3 Metodun üzerinde çalıştığı belirli nesneyi (örneği).

4 Accessor: `get_balance()`; mutator: `deposit(amount)` veya `set_balance(x)`.

5 `__str__`

6 Aynı anlama gelenler, programın ilgilenmediği öğeler, sınıf değil nesne olanlar ve basit bir değişkene sığan değerler.